KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

# Mini Attendance Tracker

SUBMITTED TO

**The Technical Department**

KG College of Arts and Science

SUBMITTED BY

| Student Name | Roll No. | Course & Batch |
|---|---|---|
| Mohit Krishna K S | 25BIT126 | B.SC IT 'B' (2025-2028) |
| Murali Karthick K | 25BIT127 | B.SC IT 'B' (2025-2028) |
| Nalinaksha M | 25BIT128 | B.SC IT 'B' (2025-2028) |
| Navya N | 25BIT129 | B.SC IT 'B' (2025-2028) |
| Netra S | 25BIT130 | B.SC IT 'B' (2025-2028) |

Academic Year: **2025 – 2026**

Semester: **II**

# DECLARATION

We hereby declare that the project entitled "Mini Attendance Tracker" submitted in partial fulfilment of the requirements for the course [**C PROGRAMMING]** is a **record of our original work** carried out under the guidance of **[VASANTH M]**, Technical Trainer, Technical Department, KGiSL MicroCollege.

We further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

We have acknowledged all sources of information used in this project wherever applicable.

| Student Name | Roll No. | Course & Batch | Signature |
|---|---|---|---|
| Mohit Krishna K S | 25BIT126 | B.SC IT 'B' (2025-2028) | |
| Murali Karthick K | 25BIT127 | B.SC IT 'B' (2025-2028) | |
| Nalinaksha M | 25BIT128 | B.SC IT 'B' (2025-2028) | |
| Navya N | 25BIT129 | B.SC IT 'B' (2025-2028) | |
| Netra S | 25BIT130 | B.SC IT 'B' (2025-2028) | |

**Place:** COIMBATORE

**Date:** _____

# CERTIFICATE

This is to certify that the project entitled **"Mini Attendance Tracker"** is a **bona fide work** carried out by the following students of **[BSC.IT "B" (2025-2028)]**, [**B.SC.INFORMATION AND TECHNOLOGY]**, **KG College of Arts & Science**, in partial fulfilment of the requirements for the course **[C PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **[MARCH,2026]**.

It is further certified that this project is the **original work** of the students and has not been previously submitted for the award of any degree, diploma, or certificate at this or any other institution.

**Project Team:**

| S. No. | Name | Roll Number |
|---|---|---|
| 1. | Mohit Krishna K S | 25BIT126 |
| 2. | Murali Karthick K | 25BIT127 |
| 3. | Nalinaksha M | 25BIT128 |
| 4. | Navya N | 25BIT129 |
| 5. | Netra S | 25BIT130 |

**Guide:**
(Technical Trainer, KGM)

**Head of the Department**
(Technical Head, KGM)

**Place:** Coimbatore

**Date:**

## ABSTRACT

The project **"Mini Attendance Tracker"** is developed using the C programming language to automate and simplify the process of recording and managing attendance. The main objective of this project is to provide an efficient and organized system for maintaining attendance records for students or employees.

The system allows the user to input names or IDs and mark attendance as present or absent. It stores the data systematically and enables easy calculation of attendance percentage. The program can display attendance reports and, if required, save the records into a text file for future reference and record maintenance.

This project demonstrates the effective use of arrays, structures, functions, and file handling in C to build a simple yet practical application. The design focuses on user-friendliness, accuracy, and clarity, making it suitable for small-scale usage such as classrooms or small organizations.

Through this project, important concepts like structured programming, control flow, modular design, and data management are implemented successfully. It enhances programming skills and provides a real-world understanding of how software can be used to manage routine tasks efficiently.

## TABLE OF CONTENTS


| S. No. | Topics | Page. No. |
|---|---|---|
| **1** | **Introduction** | |
| 1.1 | Objective | |
| 1.2 | Overview | |
| 1.3 | Features | |
| **2** | **Program Analysis** | |
| 2.1 | Problem Definition | |
| 2.2 | Project Overview | |
| 2.3 | Scopes and Limitations | |
| **3** | **System Design** | |
| 3.1 | Modules | |
| **4** | **Methodology** | |
| 4.1 | Problem Analysis | |
| 4.2 | System Design | |
| 4.3 | Program Development | |
| 4.4 | Testing and Validation | |
| 4.5 | Result and Documentation | |
| 4.6 | Algorithm | |
| 4.7 | Flowchart | |
| **5** | **Source Code** | |
| **6** | **Input and Output Screens** | |
| **7** | **Discussion and Conclusion** | |
| 7.1 | Discussion | |
| 7.2 | Conclusion | |
| **8** | **References** | |

# 1. INTRODUCTION

### 1.1 Objective

• To design and develop a Mini Attendance Tracker using the C programming language.
• To automate student attendance recording and management.
• To demonstrate structured programming concepts — arrays, loops, conditional statements, functions, and file handling (optional).

### 1.2Overview

This system allows users (teachers) to:
• View student list.
• Mark attendance (Present/Absent).
• Calculate attendance percentage.
• Store attendance records for future reference.

### 1.3 Features

• Simple menu-driven interface.
• Student data stored using arrays/structures.
• Attendance marking for multiple students.
• Calculates attendance percentage.
• Option to save attendance in a file.
• Displays summary report.

# 2. PROGRAM ANALYSIS

### 2.1 Problem Definition

A school requires a system to:
• Record daily attendance of students.
• Avoid manual errors in calculation.
• Generate attendance reports easily.

### 2.2 Project Overview

| Aspect | Description |
|---|---|
| Language Used | C |
| Compiler | GCC / Dev-C++ / Turbo C++ |
| Concepts Used | Arrays, Structures, Loops, Functions |
| I/O Type | Console-based |
| Optional | File handling |

### 2.3 Scopes and Limitations

**Scope:**
• Suitable for small classes.
• Can be extended with database.

**Limitations:**
• Console-based only.
• No GUI.
• Manual data entry required.

## 3. SYSTEM DESIGN

### 3.1 Modules

1. Student Display Module
Displays list of students.

2. Attendance Input Module
Takes attendance (Present/Absent).

3. Calculation Module
Calculates attendance percentage.

4. Report Module
Displays attendance summary.

5. File Storage Module (Optional)
Stores attendance in file.

6. Exit Module
Ends the program.

Module Flow

1. Display Students
2. Take Attendance
3. Calculate Percentage
4. Display Report
5. Save File
6. Exit

# 4. METHODOLOGY

The development of the mini attendance tracker project follows a structured methodology that includes five main stages: **Problem Analysis, System Design, Program Development, Testing & Validation, and Results & Documentation.**

### 4.1. Problem Analysis

Manual attendance is time-consuming and error-prone.
This system solves it digitally.

### 4.2. System Design

Structures used:

- Student details
- Attendance records

### 4.3. Program Development

Uses:
• Arrays
• Structures
• Loops
• Functions

### 4.4. Testing and Validation

• Tested for valid/invalid inputs

• Verified percentage calculation

### 4.5. Result and Documentation

• Accurate attendance report generated

• Easy to use and maintain

### 4.6. Algorithm

1. Start program
2. Enter number of students
3. Input student names
4. For each student:
5. Mark attendance (1 = Present, 0 = Absent)
6. Calculate attendance percentage
7. Display report
8. Save to file (optional)
9. End program

### 4.7. Flowchart

1. Start
2. Input student details
3. Mark attendance
4. Calculate percentage
5. Display report
6. Save file
7. End

# 5. SOURCE CODE

```
#include <stdio.h>
#define MAX_STUDENTS 100
#define NAME_LEN 50
#define DATE_LEN 20
struct Student {
    int roll;
    char name[NAME_LEN];
};
// ---------------- CUSTOM STRING FUNCTIONS ----------------
int str_compare(char s1[], char s2[]) {
    int i = 0;
    while (s1[i] != '\0' && s2[i] != '\0') {
        if (s1[i] != s2[i])
            return 0;
        i++;
    }
    return (s1[i] == '\0' && s2[i] == '\0');
}
void remove_newline(char s[]) {
    int i = 0;
    while (s[i] != '\0') {
        if (s[i] == '\n') {
            s[i] = '\0';
            break;
        }
        i++;
```

```
    }
}
// ---------------- STUDENT MODULE ----------------
int load_students(struct Student students[]) {
    FILE *f = fopen("students.txt", "r");
    int count = 0;
    if (!f) return 0;
    while (fscanf(f, "%d,%49[^\n]", &students[count].roll, students[count].name) == 2) {
        count++;
        if (count >= MAX_STUDENTS) break;
    }
    fclose(f);
    return count;
}
void save_student(struct Student s) {
    FILE *f = fopen("students.txt", "a");
    if (!f) { printf("Error opening students.txt\n"); return; }
    fprintf(f, "%d,%s\n", s.roll, s.name);
    fclose(f);
}
void add_student(struct Student students[], int *n) {
    if (*n >= MAX_STUDENTS) {
        printf("Limit reached!\n");
        return;
    }
    struct Student s;
    printf("Enter roll number: ");
    scanf("%d", &s.roll);
    getchar();
```

```
    printf("Enter name: ");
    fgets(s.name, NAME_LEN, stdin);
    remove_newline(s.name);
    for (int i = 0; i < *n; i++) {
        if (students[i].roll == s.roll) {
            printf("Roll number already exists!\n");
            return;
        }
    }
    students[*n] = s;
    (*n)++;
    save_student(s);
    printf("? Student added successfully!\n");
}
void list_students(struct Student students[], int n) {
    if (n == 0) {
        printf("No students found. Add students first.\n");
        return;
    }
    printf("\n--- Student List ---\nRoll\tName\n");
    for (int i = 0; i < n; i++)
        printf("%d\t%s\n", students[i].roll, students[i].name);
}
// ---------------- ATTENDANCE MODULE ----------------
void mark_attendance(struct Student students[], int n) {
    if (n == 0) {
        printf("No students to mark. Add students first.\n");
        return;
    }
```

```
    FILE *f = fopen("attendance.txt", "a");
    if (!f) { printf("Error opening attendance.txt\n"); return; }
    char date[DATE_LEN];
    printf("Enter date (DD-MM-YYYY): ");
    scanf("%s", date);
    printf("\nMark attendance (P = Present, A = Absent):\n");
    for (int i = 0; i < n; i++) {
        char status;
        printf("%d - %s (P/A): ", students[i].roll, students[i].name);
        scanf(" %c", &status);
        if (status == 'P' || status == 'p')
            fprintf(f, "%s,%d,P\n", date, students[i].roll);
        else
            fprintf(f, "%s,%d,A\n", date, students[i].roll);
    }
    fclose(f);
    printf("\n? Attendance saved for %s!\n", date);
}
// ---------------- REPORT MODULE ----------------
void student_report(struct Student students[], int n) {
    if (n == 0) { printf("No students in list.\n"); return; }
    int roll, total = 0, present = 0;
    char date[DATE_LEN], status;
    printf("Enter roll number: ");
    scanf("%d", &roll);
    FILE *f = fopen("attendance.txt", "r");
    if (!f) { printf("No attendance records found.\n"); return; }
    char line[200];
    while (fgets(line, sizeof(line), f)) {
```

```
        int r;
        sscanf(line, "%[^,],%d,%c", date, &r, &status);
        if (r == roll) {
            total++;
            if (status == 'P') present++;
        }
    }
    fclose(f);
    if (total == 0) { printf("No records found for this roll number.\n"); return; }
    double percent = (present * 100.0) / total;
    printf("\nReport for Roll No %d\n", roll);
    printf("Total Classes: %d\nPresent: %d\nAttendance: %.2f%%\n", total, present, percent);
}
void date_summary() {
    FILE *f = fopen("attendance.txt", "r");
    if (!f) { printf("No attendance data available.\n"); return; }
    char search_date[DATE_LEN], date[DATE_LEN], status, line[200];
    int roll, present = 0, absent = 0;
    printf("Enter date (DD-MM-YYYY): ");
    scanf("%s", search_date);
    while (fgets(line, sizeof(line), f)) {
        sscanf(line, "%[^,],%d,%c", date, &roll, &status);
        if (str_compare(date, search_date)) {
            if (status == 'P') present++;
            else absent++;
        }
    }
    fclose(f);
    printf("\nSummary for %s\nPresent: %d\nAbsent: %d\nTotal: %d\n",
```

```
        search_date, present, absent, present + absent);
}
// ---------------- MAIN PROGRAM ----------------
int main() {
    struct Student students[MAX_STUDENTS];
    int n = load_students(students);
    int choice;
    printf("====== MINI ATTENDANCE TRACKER - BSC IT B SECTION ======\n");
    while (1) {
        printf("\n1. Add Student\n2. List Students\n3. Mark Attendance\n4. Student Report\n5. Date Summary\n6. Exit\nEnter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: add_student(students, &n); break;
            case 2: list_students(students, n); break;
            case 3: mark_attendance(students, n); break;
            case 4: student_report(students, n); break;
            case 5: date_summary(); break;
            case 6: printf("\nExiting program... Goodbye!\n"); return 0;
            default: printf("Invalid choice! Try again.\n");
        }
    }
}
```

**SOURCE CODE EXPLANATION:**

**1) Headers & Macros**

```
#include <stdio.h>

#define MAX_STUDENTS 100
#define NAME_LEN 50
#define DATE_LEN 20
```

• stdio.h → Used for input and output functions like printf, scanf, fopen, etc.
• MAX_STUDENTS → Maximum number of students allowed in the system.
• NAME_LEN → Maximum length of student name.
• DATE_LEN → Maximum length of date string.

👉 These macros help in memory management and avoid overflow.

---

**2) Data Models (Structure)**

```
struct Student {
   int roll;
   char name[NAME_LEN];
};
```

• Stores student details:

- roll → Unique roll number
- name → Student name

👉 This structure represents each student record.

---

**3) Custom String Functions**

int str_compare(char s1[], char s2[])

• Compares two strings manually (instead of strcmp).
• Returns 1 if equal, otherwise 0.

void remove_newline(char s[])

• Removes newline (\n) from input string (used after fgets).

👉 These functions improve input handling and string comparison.

---

**4) Load Students from File**

int load_students(struct Student students[])

• Opens "students.txt" in read mode.
• Reads student data in format:

roll,name

• Stores data into array.
• Returns total number of students loaded.

👉 Helps in retrieving previously saved student records.

---

**5) Save Student to File**

void save_student(struct Student s)

• Opens "students.txt" in append mode.
• Saves new student details.

👉 Ensures data is stored permanently.

---

**6) Add Student Module**

void add_student(struct Student students[], int *n)

• Takes roll number and name as input.
• Checks for duplicate roll number.
• Adds student to array and file.

👉 Prevents duplicate entries and updates student list.

---

**7) List Students Module**

void list_students(struct Student students[], int n)

• Displays all students in tabular format:

Roll  Name

👉 Helps teacher view all registered students.

---

**8) Mark Attendance Module**

void mark_attendance(struct Student students[], int n)

• Opens "attendance.txt" in append mode.
• Takes date input.
• For each student:

- Accepts P (Present) or A (Absent)
- Saves record in file:

date,roll,P/A

👉 Example:

12-03-2026,101,P

👉 Stores daily attendance efficiently.

---

**9) Student Report Module**

void student_report(struct Student students[], int n)

• Takes roll number as input.
• Reads "attendance.txt".
• Calculates:

- Total classes
- Present count
- Attendance percentage

percent = (present * 100.0) / total;

👉 Displays individual student performance.

---

**10) Date Summary Module**

void date_summary()

• Takes a specific date.
• Counts:

- Total Present
- Total Absent

👉 Uses custom string comparison to match date.

👉 Displays summary like:

Present: 25
Absent: 5
Total: 30

## 6. INPUT AND OUTPUT SCREENS

**Input Screen :**

```
C:\Users\KGCAS\AppData\Loc

====== MINI ATTENDANCE TRACKER - BSC IT B SECTION ======

1. Add Student
2. List Students
3. Mark Attendance
4. Student Report
5. Date Summary
6. Exit
Enter your choice:
```

**Output Screen:**

```
C:\Users\KGCAS\AppData\Loc

====== MINI ATTENDANCE TRACKER - BSC IT B SECTION ======

1. Add Student
2. List Students
3. Mark Attendance
4. Student Report
5. Date Summary
6. Exit
Enter your choice: 1
Enter roll number: 12
Enter name: shifa
? Student added successfully!

1. Add Student
2. List Students
3. Mark Attendance
4. Student Report
5. Date Summary
6. Exit
Enter your choice: 2

--- Student List ---
Roll    Name
12      shifa

1. Add Student
2. List Students
3. Mark Attendance
4. Student Report
5. Date Summary
6. Exit
Enter your choice: 3
Enter date (DD-MM-YYYY): 09-07-2007

Mark attendance (P = Present, A = Absent):
12 - shifa (P/A): a

? Attendance saved for 09-07-2007!

1. Add Student
2. List Students
3. Mark Attendance
4. Student Report
5. Date Summary
6. Exit
```

# 7. DISCUSSION & CONCLUSION

### 7.1 Discussion

- Reduces manual effort
- Ensures accuracy
- Easy to use

### 7.2 Conclusion

- Successfully tracks attendance
- Demonstrates C programming concepts
- Can be extended with database or GUI

## 8. REFERENCES


- Kernighan & Ritchie – *The C Programming Language*
- GeeksforGeeks – C Programming
- TutorialsPoint – C Basics
- Dev-C++ Documentation